
Introducción al Aprendizaje Automático

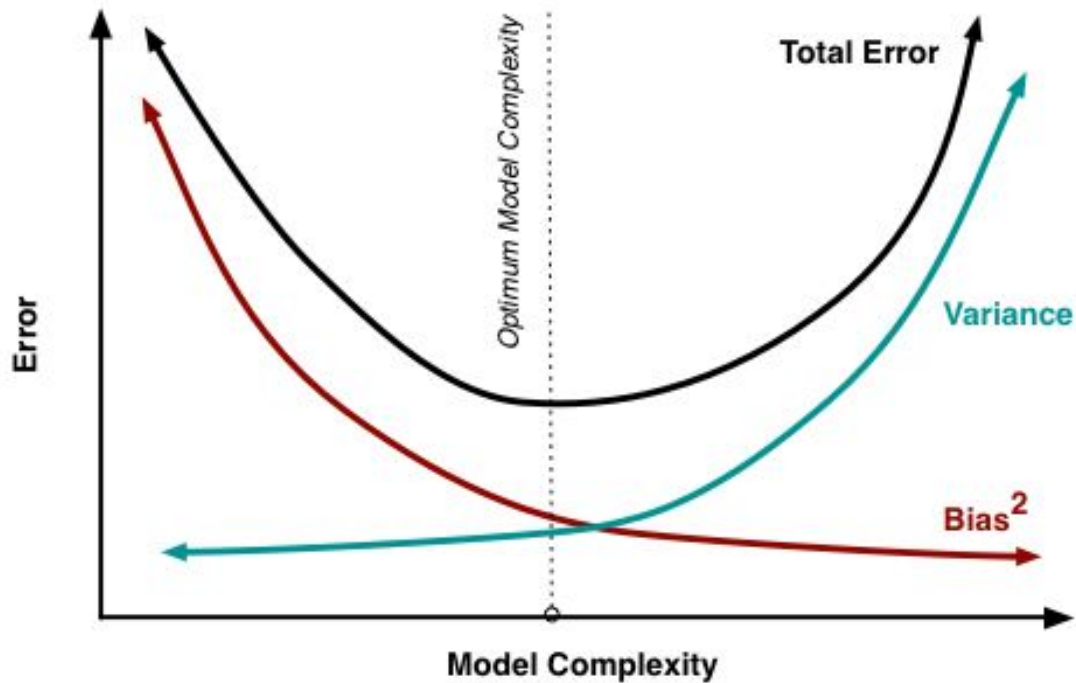
1er cuatrimestre 2025



Universidad
Nacional
de San Martín

Ensembles

Trade-off entre sesgo y varianza



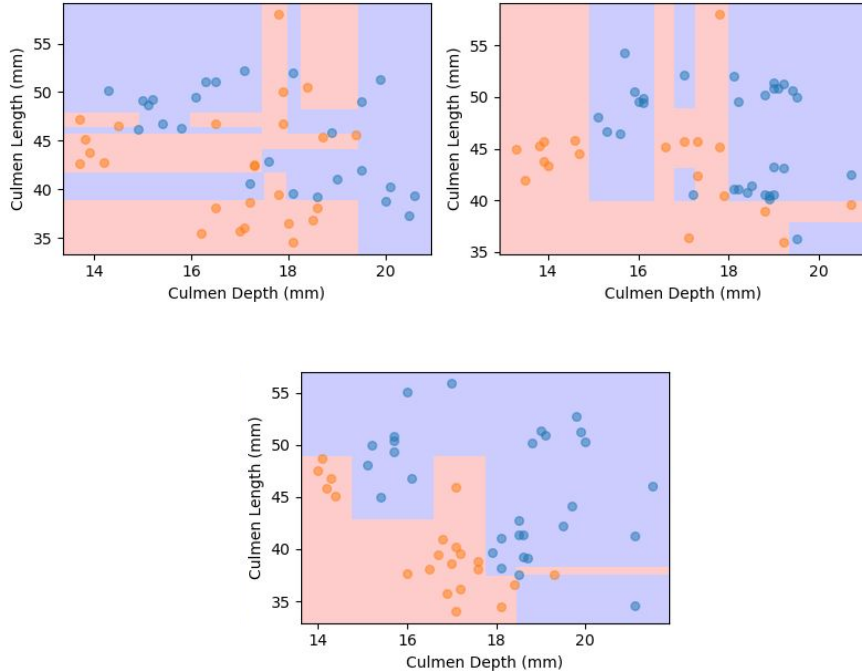
ALTA VARIANZA – BAJO BIAS



Los algoritmos de bajo bias (alta varianza) tienden a ser **más complejos**, con una estructura subyacente flexible.

¿Podremos usar estos modelos para mejorar las predicciones?

ALTA VARIANZA – BAJO BIAS



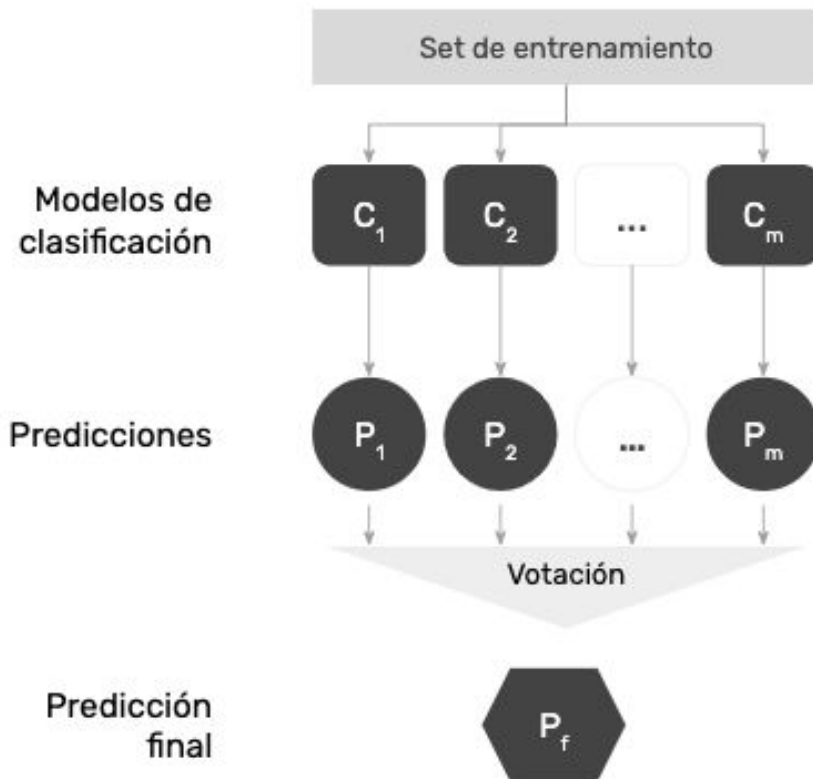
Los algoritmos de bajo bias (alta varianza) tienden a ser **más complejos**, con una estructura subyacente flexible.

¿Podremos usar estos modelos para mejorar las predicciones?

Ensamblés

Idea: entrenar muchos modelos y hacerlos votar. La clasificación resultante es la que reciba más votos.

Aún mejor, si los modelos devuelven scores, se puede hacer una votación ponderada.



Ensambles

¿Qué necesitamos para que esta idea funcione?

Si todos los modelos son muy parecidos, no van a agregar mucha información nueva en la votación.

Necesitamos modelos diferentes entre sí, poco correlacionados.

Los modelos pueden ser diferentes entre sí por una variedad de razones:

- Puede haber diferencia en la **población de datos**
- Puede haber una **técnica de modelado** utilizada diferente
- Puede haber una **hipótesis** diferente

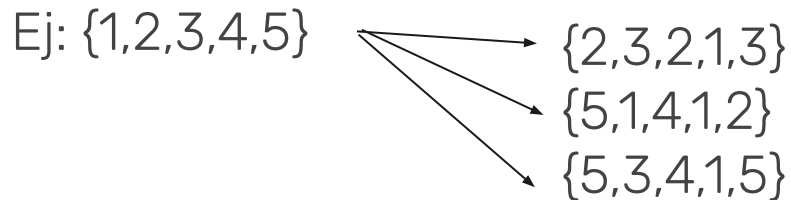
Existen varias técnicas para generar modelos de ensambles.
Las más conocidas son:

BAGGING (Bootstrap Aggregation).
BOOSTING.
STACKING.

Bagging o Bootstrap Aggregation

El Bagging es una de las técnicas de construcción de conjuntos que también se conoce como Agregación Bootstrap.

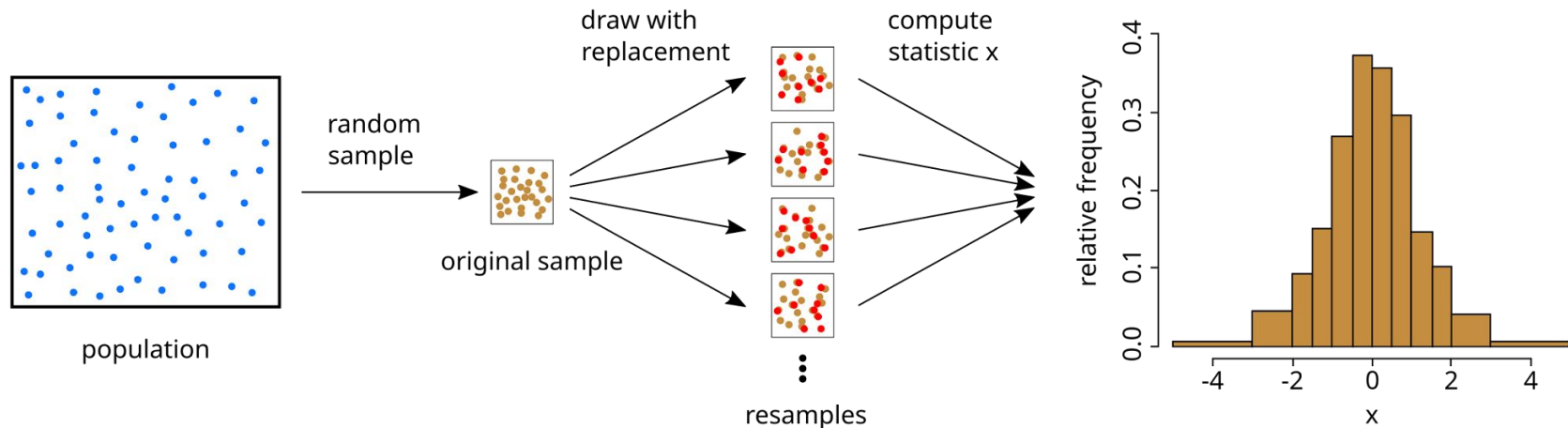
¿Qué es Bootstrap? Es una técnica de muestreo con reemplazo.



Bootstrap

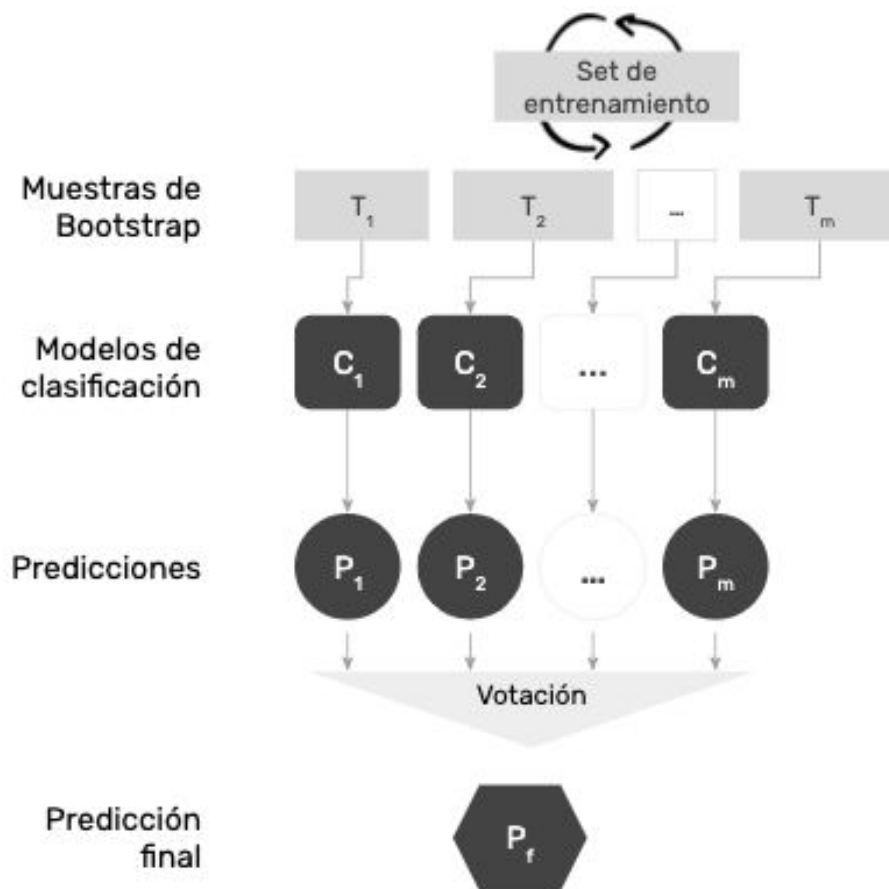
"El bootstrapping es un procedimiento para estimar la distribución de un estimador mediante el remuestreo (a menudo con reemplazo) de los datos o un modelo estimado a partir de los datos." **Wikipedia**

"El bootstrap es una herramienta estadística ampliamente aplicable y extremadamente poderosa que puede utilizarse para cuantificar la incertidumbre asociada con un estimador dado o un método de aprendizaje estadístico." **An Introduction to Statistical Learning**



Bagging

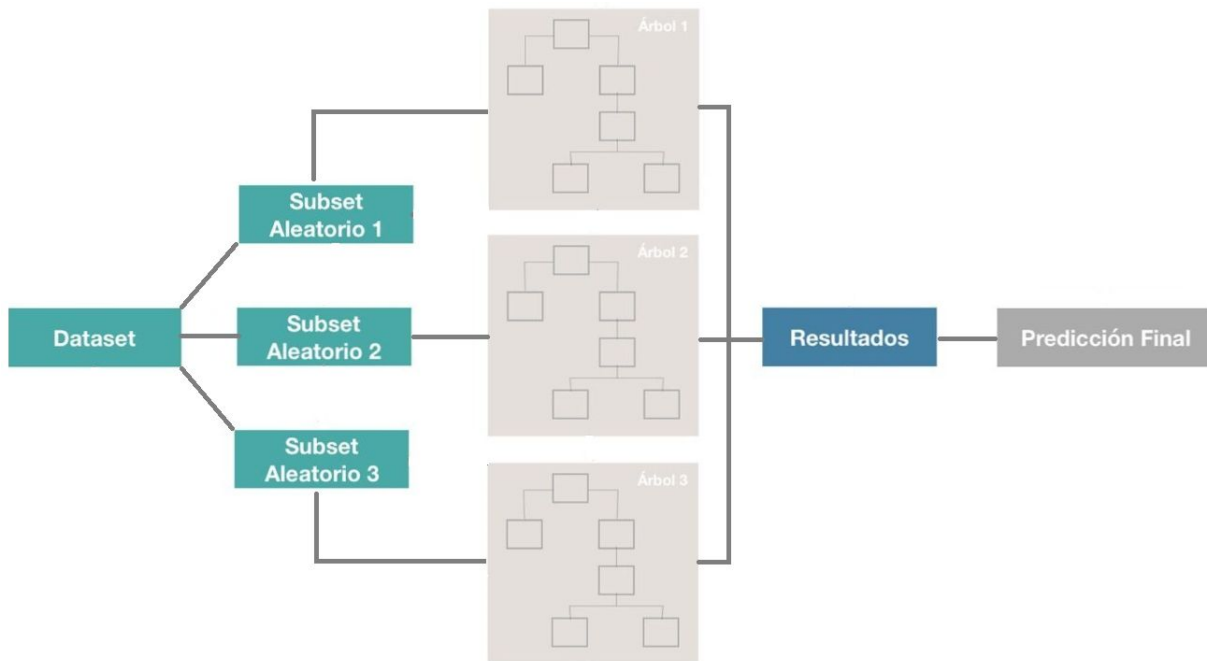
- Dada una muestra de datos, se extraen varias muestras, *bootstrapped*
- Esta selección se realiza de manera aleatoria.
- Una vez que forman las muestras *bootstrapped*, se entrenan los modelos de manera separada. En general, estos modelos serán modelos con mucha varianza.
- La predicción de salida final se combina en las proyecciones de todos los submodelos.



Bagging o Bootstrap Aggregation

Esta técnica se puede usar con cualquier tipo de modelo: Árboles, KNN, SVM, etc.

Pero **lo más común** es que se aplique en árboles, para crear bosques.



Bagging o Bootstrap Aggregation

Esta técnica se puede usar con cualquier tipo de modelo: Árboles, KNN, SVM, etc.

Pero **lo más común** es que se aplique en árboles, para crear bosques.

¿Por qué árboles?

Pros:

- Los árboles de decisión reflejan (aprox) fielmente la toma de decisiones humanas.
- Los árboles pueden mostrarse gráficamente y son fácilmente interpretables incluso por alguien sin experiencia (especialmente si son pequeños).
- Los árboles pueden manejar fácilmente atributos categóricos sin la necesidad de crear variables ficticias (dummy) (esto depende de la implementación).
- Son fácilmente entrenables.

Contras:

- Los árboles generalmente no tienen el mismo nivel de desempeño de otros modelos.
- Los árboles pueden ser muy poco robustos. Un pequeño cambio en los datos puede provocar un gran cambio en el árbol estimado finalmente.

Random Forest

- Ensemble learning
- Bagging
- Boosting
- Random Forest
- Gradient Boosting
- XGBoost
- LightGBM
- CatBoost

Random Forest

Problema: si pocos atributos (features) son predictores fuertes, todos los árboles se van a parecer entre sí. Esos atributos terminarán cerca de la raíz para todos los conjuntos generados con bootstrap.

Random Forest

Problema: si pocos atributos (features) son predictores fuertes, todos los árboles se van a parecer entre sí. Esos atributos terminarán cerca de la raíz para todos los conjuntos generados con bootstrap.

Random Forest es igual a bagging, pero en cada nodo, hay que considerar sólo un subconjunto de m atributos elegidos al azar (random feature selection)

¿Cómo funciona Random Forest?

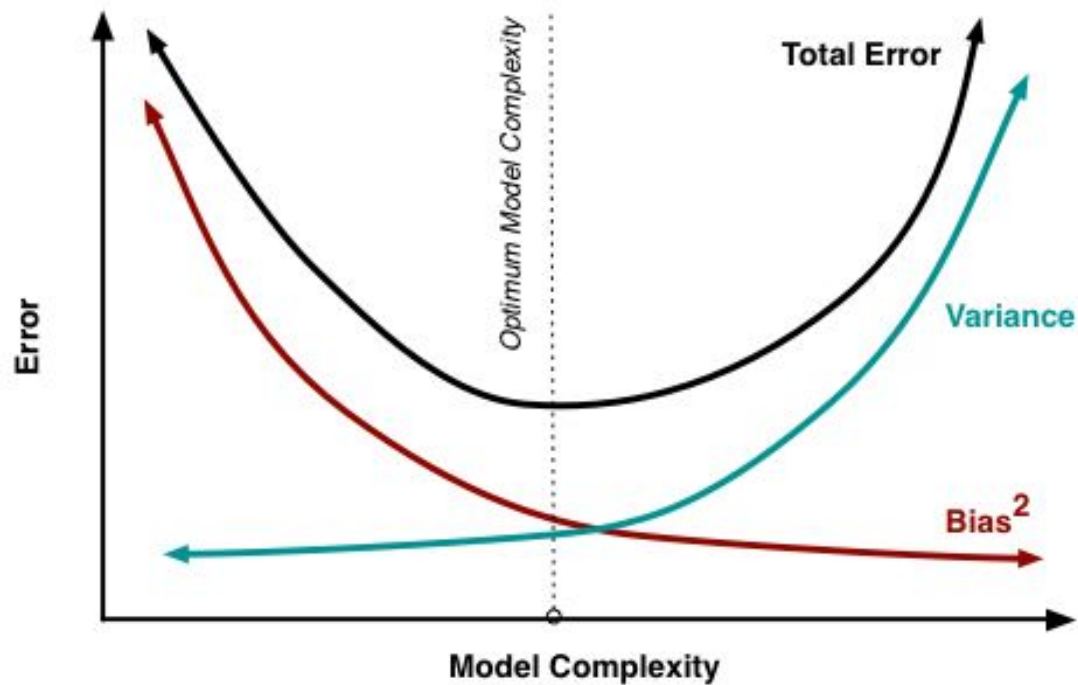
- Se seleccionan **k features de las m totales** (siendo k menor a m) y se crea un árbol de decisión con esas k features.
- Se crean **n árboles** variando siempre la cantidad de **k features**
- Se guarda el resultado de cada árbol obteniendo **n salidas.**
- Se calculan los votos obtenidos para cada “clase” seleccionada y se considera a la más votada como la clasificación final de nuestro “bosque”.

¿Qué pasa cuando $k = m$?

Random Forest • Ventajas

1. Bastante robusto frente a outliers y ruido
2. Provee buenos estimadores de error (oob_score) e importancia de variables
3. Si bien entrenar muchos árboles puede llevar mucho tiempo, es fácilmente paralelizable.

Trade-off entre sesgo y varianza

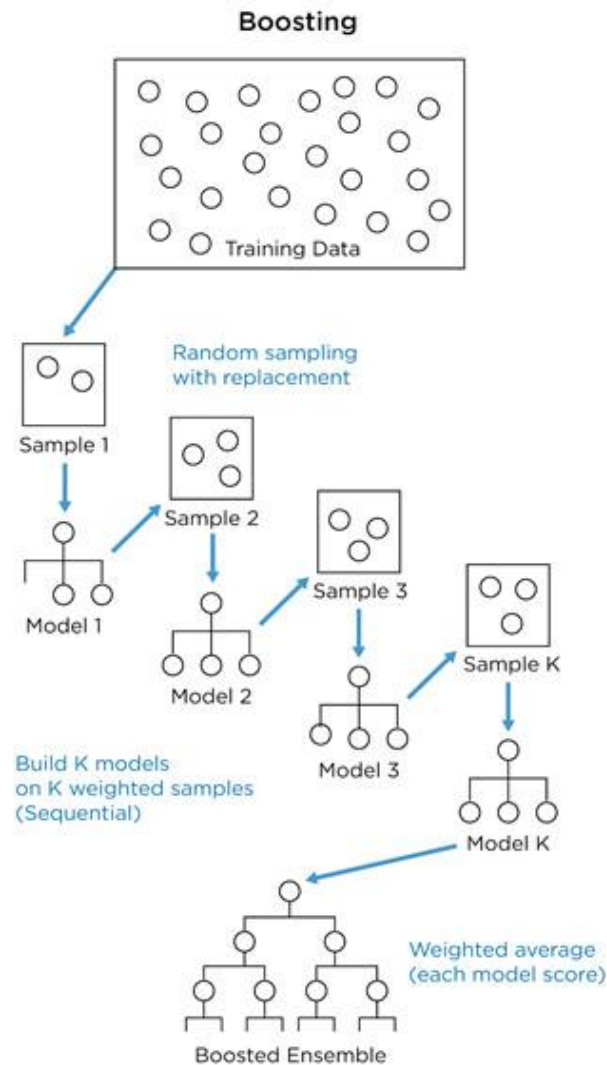


Ensembles: Boosting

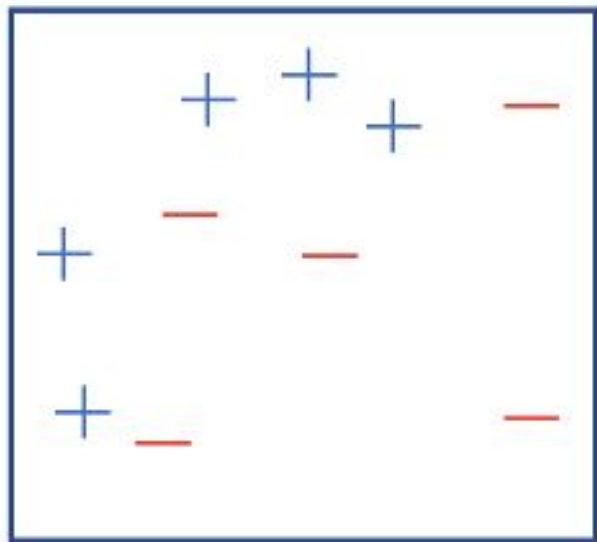
Ensambls • Boosting

Se entrena una secuencia de modelos donde se da más peso a los ejemplos que fueron clasificados erróneamente por iteraciones anteriores.

Al igual que con bagging, las tareas de clasificación se resuelven con una mayoría ponderada de votos, y las tareas de regresión se resuelven con una suma ponderada para producir la predicción final.

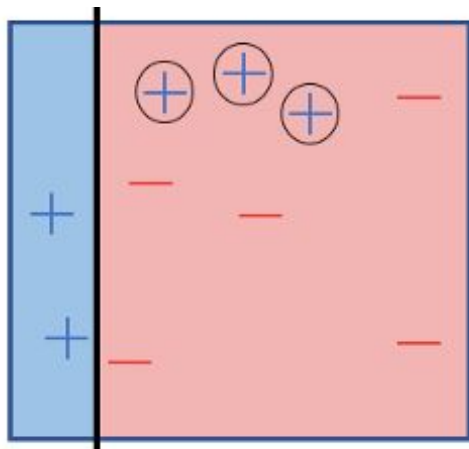


BOOSTING: AdaBoost (1 / 5)



Para explicar un poco el funcionamiento de **AdaBoost** consideremos el siguiente problema de clasificación binaria con 10 ejemplos de entrenamiento, **5 positivos y 5 negativos.**

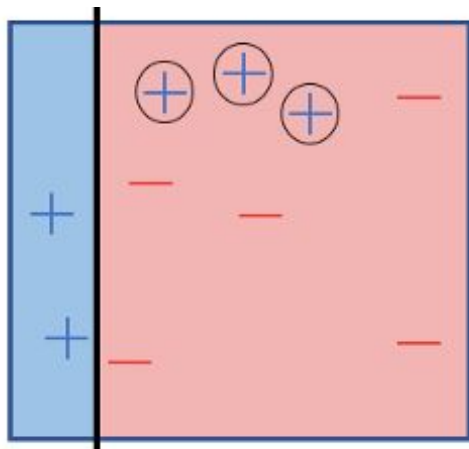
BOOSTING: AdaBoost (2 / 5)



A la izquierda el primer clasificador débil, una recta vertical. A la derecha de la recta, consideramos que todos los ejemplos son negativos, mientras que a la izquierda son positivos.

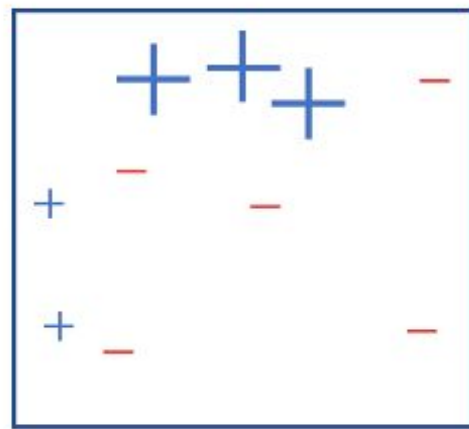
La recta clasifica mal a tres positivos.

BOOSTING: AdaBoost (2 / 5)



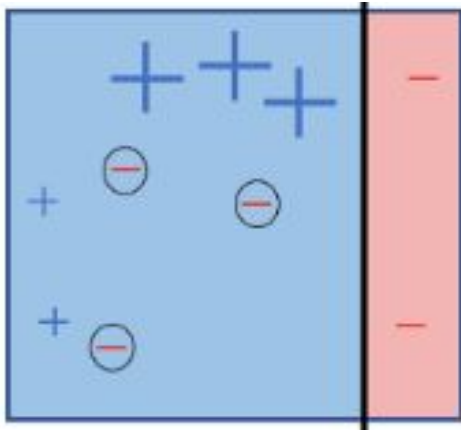
A la izquierda el primer clasificador débil, una recta vertical. A la derecha de la recta, consideramos que todos los ejemplos son negativos, mientras que a la izquierda son positivos.

La recta clasifica mal a tres positivos.



Aquí vemos como los tres ejemplos mal clasificados aparecen ahora de un mayor tamaño que el resto de los ejemplos. Esto simboliza que dichos ejemplos tendrán una mayor importancia al momento de seleccionar el clasificador débil de la segunda iteración.

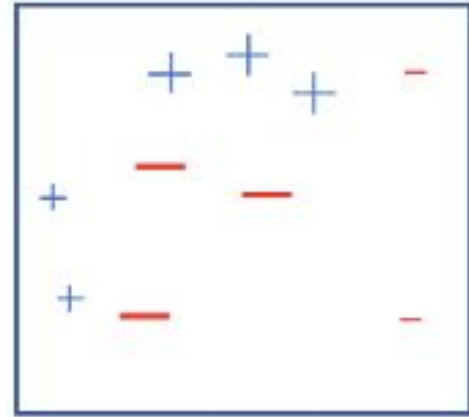
BOOSTING: AdaBoost (3 / 5)



Aquí vemos al segundo clasificador débil, que es otra recta vertical colocada más hacia la derecha.

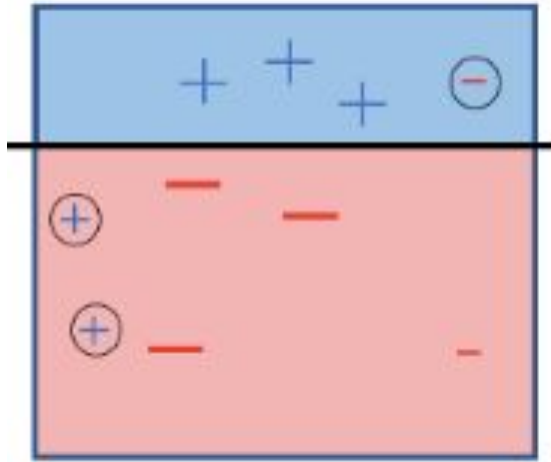
Este segundo clasificador se equivoca también en tres ejemplos,

ya que clasifica mal ejemplos negativos.



Aquí vemos que para la tercera iteración los ejemplos negativos mal clasificados tienen ahora el mayor tamaño, es decir, tendrán mayor importancia en la siguiente iteración.

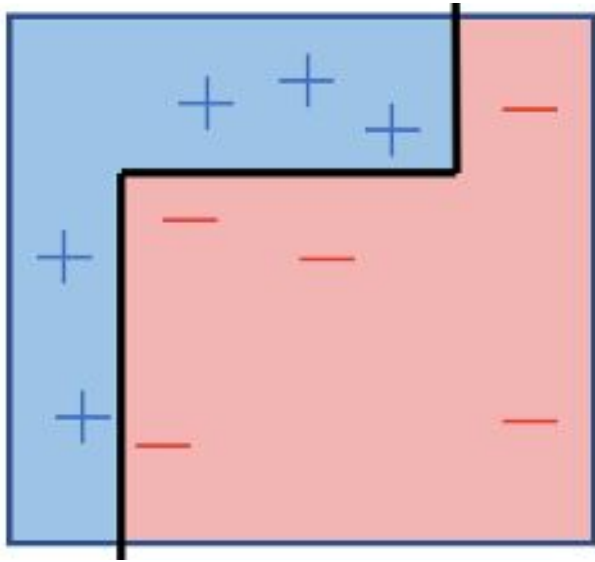
BOOSTING: AdaBoost (4 / 5)



En la tercera iteración el clasificador débil resultante es una recta horizontal, como se puede observar en el cuadro de la derecha.

Este clasificador se equivoca en la clasificación de un ejemplo negativo y dos positivos, que de igual forma aparecen encerrados en un círculo.

BOOSTING: AdaBoost (5 / 5)



Finalmente, se ilustra el clasificador fuerte que resulta de crear un ensamble con tres clasificadores débiles. La forma en que utilizamos estos tres clasificadores débiles es mediante una decisión por mayoría.

Cuando deseamos clasificar un nuevo ejemplo, le preguntamos a cada uno de nuestros tres clasificadores débiles su opinión. Si la mayoría opina que el nuevo ejemplo es positivo, pues entonces la decisión del clasificador fuerte será que es un ejemplo positivo.

Adaboost con



The following example shows how to fit an AdaBoost classifier with 100 weak learners:

```
>>> from sklearn.model_selection import cross_val_score
>>> from sklearn.datasets import load_iris
>>> from sklearn.ensemble import AdaBoostClassifier

>>> iris = load_iris()
>>> clf = AdaBoostClassifier(n_estimators=100)
>>> scores = cross_val_score(clf, iris.data, iris.target, cv=5)
>>> scores.mean()
0.9...
```

The number of weak learners is controlled by the parameter `n_estimators`. The `learning_rate` parameter controls the contribution of the weak learners in the final combination. By default, weak learners are decision stumps. Different weak learners can be specified through the `base_estimator` parameter. The main parameters to tune to obtain good results are `n_estimators` and the complexity of the base estimators (e.g., its depth `max_depth` or minimum required number of samples to consider a split `min_samples_split`).

¡A leer la documentación!

Boosting XG Boost

(Extreme Gradient Boosting)

Boosting: XGBoost (Extreme Gradient Boosting)

- XGBoost es un algoritmo que recientemente ha **dominado el aprendizaje automático** y sobre todo las competiciones de Kaggle (para datos estructurados).
- XGBoost es una **implementación de árboles de decisión potenciados por el algoritmo de descenso por gradiente**, diseñado para aumentar la velocidad y mejorar el rendimiento.
- XGBoost es una **librería de software que se puede descargar e instalar** y luego acceder desde una variedad de interfaces: CLI, C++, **Python**, R, Julia, etc.
- No sólo tiene **buena performance computacional**, también posee un **muy buen desempeño con el manejo de los datos**.

Boosting: XGBoost (Extreme Gradient Boosting)

- XGBoost es un algoritmo que recientemente ha **dominado el aprendizaje automático** y sobre todo las competiciones de Kaggle (para datos estructurados).
- XGBoost es una **implementación de árboles de decisión potenciados por el algoritmo de descenso por gradiente**, diseñado para aumentar la velocidad y mejorar el rendimiento.
- XGBoost es una **librería de software que se puede descargar e instalar** y luego acceder desde una variedad de interfaces: CLI, C++, **Python**, R, Julia, etc.
- No sólo tiene **buena performance computacional**, también posee un **muy buen desempeño con el manejo de los datos**.

Se puede usar con Python, **como si fuera un modelo de Scikit-Learn.**

Características principales

- **Paralelización** de la construcción de árboles utilizando todos los núcleos de la CPU durante el entrenamiento.
- **Computación distribuida** para el entrenamiento de modelos muy grandes utilizando clusters de máquinas.
- **Computación "fuera de núcleo"** para conjuntos de datos muy grandes que no caben en la memoria.
- **Optimización de caché** de estructuras de datos y algoritmos para aprovechar al máximo el hardware.

XG Boost en Python

Instalación: `sudo pip install xgboost`

Ejemplos: <https://github.com/tqchen/xgboost/tree/master/demo/guide-python>

```
diabetes = load_diabetes()

X = diabetes.data
y = diabetes.target

xgb_model = xgb.XGBRegressor(objective="reg:linear", random_state=42)

xgb_model.fit(X, y)

y_pred = xgb_model.predict(X)
```

¿Qué es mejor,
Bagging o Boosting?

Depende.

Bagging

- Modelos entrenados de manera independiente.
- Resuelve promediando los N modelos.
- Enfocado en reducir la varianza. Ayuda a prevenir overfitting.
- Se suele usar con modelos de bajo Sesgo y alta varianza.
- Fácilmente paralelizable.

vs.

Boosting

- Bastante Modelos entrenados enfocados en mejorar las fallas de los anteriores.
- Promedio pesado de los N modelos (su peso depende de su performance).
- Enfocado en reducir el Sesgo. En casos puede causar overfitting.
- Se suele usar con modelos de baja varianza y alto sesgo.
- No se puede paralelizar fácilmente.